

StockEasy — Software Validation Protocol (v1)

Field	Value
System	StockEasy — Simple Inventory for Sierra Leone SMEs
Document type	Software validation protocol
Version	1.0
Related documents	validation-checklist.md , sop-v1.md , training-guide-v1.md

1. Purpose

This protocol defines **how** StockEasy is validated before go-live and after significant changes. It ensures the system meets business requirements for sales, inventory, receipts, offline use, and end-of-day controls in a real shop environment.

The detailed test cases live in the [Validation checklist](#). This protocol governs planning, execution, evidence, and approval.

2. Scope

In scope

- Web application (PWA) on mobile and desktop browsers
- Production deployment (e.g. Vercel) and local development smoke checks
- Functional flows: products, sell, returns, credit book, receipts, labels, day close
- Offline queue, sync, and conflict handling
- 58mm thermal receipt and label printing (browser print path)

Out of scope (unless explicitly added to a release)

- Roarbyte / OTP provider uptime (monitored separately)
- Third-party printer drivers and paper hardware beyond browser print dialog
- Penetration testing and formal security certification
- Load testing at national scale

3. Roles and responsibilities

Role	Responsibility
Product owner / shop owner	Defines acceptance criteria; approves go-live
Validator / tester	Executes checklist; records results and defects
Developer / maintainer	Fixes defects; provides build version and release notes
Cashier (UAT)	Performs training-guide scenarios on real devices

4. Validation levels

Level	When	Environment	Minimum evidence
L1 — Developer smoke	Every merge or pre-deploy	Localhost (npm run dev)	Build passes; critical paths manual smoke
L2 — Staging / production build	Before each production deploy	Production URL + clean browser profile	Deploy log green; L3 subset on live URL
L3 — Pilot shop UAT	First shop go-live; major releases	Shop devices + printers	Completed checklist + sign-off
L4 — Regression	After fixes to offline, sync, receipts, payments	Same as L3	Re-run affected checklist sections only

5. Entry criteria (start validation)

Validation may begin when:

1. **Build succeeds** — `prisma generate`, migrations applied, `next build` completes (see Vercel build logs).
2. **Environment variables** are set on production: `DATABASE_URL`, `JWT_SECRET` (32+ characters), auth keys.
3. **Release identifier** is recorded (git commit hash or deployment URL).
4. **Test devices** are available (same models staff will use).
5. **Validator** has access to the [validation checklist](#) (printed or PDF).

6. Exit criteria (approve release / go-live)

Go-live is approved when **all** of the following are true:

1. **No open Critical or High defects** (see Section 8).
2. **Validation checklist** — all **Must pass** items (Section 7) marked Pass or justified N/A.
3. **Offline path verified on production URL** — sign in online → cache data → complete at least one sale offline → sync when online.
4. **Owner sign-off** on checklist (Section 4 of checklist).
5. **SOP and training** delivered or scheduled for staff.

7. Must-pass requirements (traceability)

These map to checklist sections. All are required for L3 pilot sign-off.

ID	Requirement	Checklist section
MP-01	Secure sign-in and session on production	A
MP-02	Product CRUD and stock adjust	B
MP-03	Sale checkout and stock deduction	D
MP-04	Return linked to original sale	E
MP-05	Offline sale queues and syncs	G
MP-06	Receipt print (58mm settings documented)	D, I

MP-07	Day close save while online	H
MP-08	Credit book payment reduces balance	F

Optional but recommended: barcode/labels (C), analytics (not in checklist — add if needed).

8. Defect classification and handling

Severity	Definition	Action
Critical	Cannot sell, data loss, wrong totals, security breach	Block release; fix before go-live
High	Offline/sync broken on production; cannot print receipts	Block release unless workaround documented
Medium	UI issues, non-blocking errors, partial offline	Fix in next release; document workaround
Low	Cosmetic, copy, nice-to-have	Backlog

Defect log (copy per issue):

ID	Date	Severity	Summary	Steps to reproduce	Expected	Actual	Status	Fix version
DEF-001							Open / Fixed	

9. Validation procedure (step-by-step)

Phase A — Preparation (30–60 min)

1. Record deployment URL, date, commit/build ID.
2. Confirm Neon/database awake; login works on production.
3. Clear old service worker if testing offline after a deploy (Application → Service Workers → Unregister).
4. Configure 58mm print defaults on test printer (checklist Section 2).
5. Assign tester names on checklist cover sheet.

Phase B — Online functional tests (2–3 hours)

1. Execute checklist sections **A → I** while **online**.
2. Capture screenshots for any Fail.
3. Log defects in Section 8 table.

Phase C — Offline and sync tests (1–2 hours)

1. While **online**: open **Sell, Products, History** (warms cache).
2. Enable airplane mode or DevTools → Offline.
3. Execute **G1–G3** and complete a sale (receipt should show; no generic “StockEasy is offline” full-page replace during checkout).
4. Restore network; verify sync (G2), resolve conflicts if any (G4).

Phase D — Production vs localhost parity

Check	Localhost	Production
Login OTP		
Sell + receipt		
Offline sale	N/A (SW often disabled in dev)	Required
PWA install prompt	N/A	Optional

Document any intentional differences in release notes.

Phase E — Review and sign-off

1. Review defect log; retest fixes.
2. Complete checklist sign-off (owner, tester, date).
3. Store completed checklist (PDF/scan) with deployment record.

10. Evidence to retain

Keep for each validated release:

- Completed [validation-checklist.md](#) (PDF or scan)
- Defect log (if any)
- Deployment URL and build/commit ID
- List of known limitations (from checklist Section 5)
- Training completion sheet (from [training-guide-v1.md](#))

11. Re-validation triggers

Full L3 validation is required when:

- Offline sync, service worker, or IndexedDB logic changes
- Checkout, receipts, payments, or day-close logic changes
- Database schema migrations affecting sales or inventory
- Auth/session or multi-tenant isolation changes

Partial re-validation (affected checklist sections only) is enough for:

- UI-only changes
- Copy/branding
- Non-critical bug fixes with no data-path change

12. Approval record

Role	Name	Signature	Date
Validator / tester			
Developer / maintainer			
Product owner / shop owner			

Release approved for go-live: Yes / No

Deployment URL / version: _____

Notes / waivers: _____

Document history

Version	Date	Author	Changes
1.0			Initial protocol